GUÍA DE LABORATORIO 11

Introducción profesional a React

Componentes, props, estado y eventos con Vite y TypeScript

En este laboratorio construirás un explorador interactivo de iniciativas académicas ficticias. La aplicación se organizará en componentes reutilizables, recibirá datos mediante props, conservará preferencias en estado local y actualizará la interfaz de forma declarativa ante cada interacción.

La guía explica primero el modelo mental de React y después desarrolla el proyecto archivo por archivo. Cada decisión se relaciona con una evidencia visible, una prueba o un criterio de calidad para que no te limites a copiar código: comprenderás qué cambia, quién es responsable y por qué React vuelve a renderizar una parte de la interfaz.

Todos los nombres, categorías e iniciativas del caso son ficticios y se emplean exclusivamente con fines educativos.

1 Resultados y organización del laboratorio

La semana 11 inicia la unidad de React después del bloque de Angular. El objetivo no es trasladar mecánicamente decoradores, plantillas o servicios, sino comprender un modelo declarativo basado en funciones, composición y flujo de datos. React es una biblioteca para construir interfaces; Vite proporciona el entorno de desarrollo y compilación. [R1] [R12]

Logro propuesto. Al finalizar, crearás una aplicación React con TypeScript que descompone la interfaz en componentes, transmite información mediante props, responde a eventos, conserva estado local sin mutarlo, deriva una lista visible y presenta estados accesibles. La compilación de producción y 18 pruebas automatizadas deberán finalizar correctamente.

Conocimientos previos

1. JavaScript moderno: módulos ES, funciones flecha, desestructuración, arreglos, map, filter y Set.
2. TypeScript básico: interfaces, tipos unión, readonly y parámetros tipados.
3. HTML semántico, CSS adaptable, formularios, eventos y accesibilidad elemental.
4. Node.js, npm, package.json, terminal y lectura de mensajes de error.

Resultados observables

Resultado	Evidencia verificable
Explicar el modelo declarativo	Relacionas datos, renderizado, eventos y nuevo renderizado con una interacción real.
Crear componentes	Cada función tiene nombre en PascalCase, una responsabilidad y marcado válido.
Usar props	Los componentes hijos reciben datos y callbacks tipados sin modificar sus entradas.
Gestionar estado	Búsqueda, categoría, favoritos y opción de visualización cambian mediante setters.
Renderizar colecciones	map crea tarjetas con id estable y filter deriva la colección visible.
Representar alternativas	La interfaz diferencia resultados, estado vacío y favorito seleccionado.
Comprobar calidad	npm run verify produce build correcto y 18 pruebas aprobadas.

Distribución sugerida

Sesión	Actividad
21	Modelo de React, JSX, componentes y props
21	Creación del proyecto y datos tipados
21	Tarjetas, composición y listas
21	Punto de control y reflexión

Sesión	Actividad
22	Estado, eventos y controles
22	Filtros, favoritos y renderizado condicional
22	Pruebas, accesibilidad y recorrido manual
22	Reto individual y cierre

2 Modelo mental de React

De datos a interfaz

En JavaScript imperativo podrías localizar un elemento y cambiar su contenido de forma manual. En React describes cómo debe verse la interfaz para un conjunto de datos. Cuando el estado cambia, React vuelve a ejecutar los componentes afectados, compara el resultado y confirma los cambios necesarios en el DOM. El código expresa el resultado deseado, no una secuencia de modificaciones de nodos.

Momento	Dato	Descripción de la interfaz
Renderizado inicial	query vacío y categoría Todas	Se muestran seis tarjetas.
Evento	El usuario escribe React	onChange solicita actualizar query.
Nuevo renderizado	query vale React	filter devuelve dos iniciativas y map crea dos tarjetas.
Confirmación	DOM actualizado	La pantalla y el resumen reflejan dos resultados.

Cuatro ideas que guían el laboratorio

Idea	Pregunta útil	Aplicación en el proyecto
Componente	¿Qué parte tiene identidad y responsabilidad propias?	Hero, FilterBar, InitiativeCard, SummaryPanel y EmptyState.
Props	¿Qué necesita recibir el hijo?	Una tarjeta recibe iniciativa, favorito y un callback.
Estado	¿Qué cambia por interacción y debe recordarse?	Texto, categoría, opción y conjunto de ids favoritos.
Derivación	¿Puede calcularse desde datos existentes?	La lista visible se calcula; no se almacena como otro estado.

React y Vite cumplen papeles distintos

React y react-dom suministran componentes, Hooks y renderizado para el navegador. Vite inicia el servidor de desarrollo, transforma TSX y crea el paquete de producción. TypeScript comprueba tipos; Bootstrap ofrece clases CSS; Vitest ejecuta pruebas. Ninguna herramienta reemplaza a las demás.

3 JSX y TSX sin misterios

Qué es JSX

JSX es una extensión de sintaxis que permite escribir marcado parecido a HTML dentro de JavaScript. Una herramienta de construcción lo transforma antes de que el navegador lo ejecute. Cuando el archivo además utiliza TypeScript, la extensión habitual es .tsx. JSX no es una cadena, no es HTML copiado literalmente y no constituye un lenguaje de plantillas separado. [R2] [R14]

Reglas esenciales

Regla	Ejemplo o motivo
Un único nodo raíz	Agrupa elementos con un contenedor o un fragmento <>...</>.
Cerrar etiquetas	input se escribe <input /> y cada apertura tiene cierre.
Propiedades en camelCase	className reemplaza class; onClick reemplaza onclick.
Expresiones entre llaves	{initiative.title} evalúa JavaScript y presenta el resultado.
Objetos con doble llave	style={{ color: value }} pasa un objeto a la prop style.
Comentarios especiales	{/* explicación */} es un comentario dentro del marcado.

Expresión no significa instrucción

Dentro de llaves puedes usar una expresión que produzca un valor: una variable, una llamada, una suma o un operador condicional. No insertas directamente un if, for o declaración const dentro del marcado. Prepara el valor antes del return o utiliza expresiones adecuadas como map y el operador ternario.

Ejemplo didáctico de expresiones en TSX

```
const title = initiative.title;
const hoursLabel = `${initiative.durationHours} horas`;

return (
  <h3>{title}</h3>
);
```

Atributos y accesibilidad

Las propiedades aria-* y data-* conservan guiones. React permite escribir aria-pressed y aria-label de forma equivalente al atributo web. El botón favorito usa aria-pressed para comunicar un estado alternable y un aria-label que incluye el título de la iniciativa. [R15]

4 Componentes y composición

Un componente es una función

Un componente React es una función JavaScript que devuelve marcado. Su nombre comienza con mayúscula para distinguirlo de una etiqueta nativa. Puede recibir un único objeto de props, ejecutar cálculos puros y componer otros componentes. [R3]

Ejemplo mínimo de componente tipado

```
interface GreetingProps {
  readonly name: string;
}

function Greeting({ name }: GreetingProps) {
  return <p>Hola, {name}</p>;
}
```

Responsabilidad y tamaño

No existe un número universal de líneas para separar un componente. Extrae uno cuando representa una unidad reconocible, se repite, posee comportamiento propio o ayuda a entender al componente padre. Evita componentes sin propósito que solo mueven una etiqueta a otro archivo.

Componente	Responsabilidad	No decide
Hero	Presentar título y cantidad total.	Filtros ni favoritos.
FilterBar	Presentar controles y emitir cambios.	Qué iniciativas coinciden.
InitiativeCard	Presentar una iniciativa y emitir una intención.	Cómo se guarda el estado global de la vista.
SummaryPanel	Comunicar cantidades.	Cómo se calculan.
EmptyState	Explicar ausencia de resultados y ofrecer recuperación.	Qué filtro produjo el vacío.
App	Poseer estado, derivar datos y componer la página.	Detalles visuales internos de cada tarjeta.

Componentes puros

Con las mismas props y el mismo estado, un componente debe describir la misma salida. No modifiques variables externas durante el renderizado ni ejecutes una acción inesperada en el cuerpo del componente. StrictMode ayuda a revelar algunos cálculos impuros durante el desarrollo. [R11]

5 Props flujo de datos y callbacks

Props son entradas de solo lectura

El componente padre configura al hijo mediante props. React entrega una instantánea para ese renderizado; el hijo no debe reasignar ni modificar el objeto recibido. Si necesita solicitar un cambio, llama un callback proporcionado por el padre. [R4]

Prop de InitiativeCard	Tipo	Uso
Initiative	Initiative	Datos que se presentan.
isFavorite	boolean	Determina icono, clase y aria-pressed.
onToggleFavorite	(initiativeId: number) => void	Comunica la intención al padre.

Flujo descendente y comunicación ascendente

1. App posee favoriteIds y calcula si cada id pertenece al conjunto.
2. App pasa isFavorite y handleToggleFavorite a cada InitiativeCard.
3. La tarjeta recibe un clic y llama onToggleFavorite con su id.
4. App crea un Set nuevo y solicita el cambio de estado.
5. React vuelve a renderizar App y la tarjeta recibe el nuevo valor.

Desestructuración

La desestructuración en el parámetro permite nombrar cada prop. El tipo de props documenta el contrato y evita omisiones o valores incompatibles. readonly comunica intención al compilador, aunque la disciplina principal sigue siendo no mutar entradas durante el renderizado.

6 Estado local eventos y nuevo renderizado

Estado es memoria del componente

useState declara una variable de estado y una función que solicita su actualización. El valor persiste entre renderizados. Un cambio mediante el setter programa otro renderizado; modificar directamente una estructura existente no expresa ese cambio de forma segura. [R8]

Dos estados con tipos diferentes

```
const [query, setQuery] = useState('');
const [favoriteIds, setFavoriteIds] = useState<Set<number>>(() => new Set());
```

Reglas de Hooks

1. Llama Hooks únicamente en el nivel superior de un componente o de un Hook personalizado.
2. No llames useState dentro de if, bucles, manejadores ni funciones anidadas.
3. Usa el prefijo use para Hooks personalizados; una función común no debe fingir que es Hook.
4. Conserva un orden de llamadas estable en cada renderizado. [R9]

Actualización basada en el valor anterior

El siguiente conjunto depende del conjunto actual. Por eso se emplea la forma funcional del setter. toggleFavorite copia el Set, cambia la copia y la devuelve. La referencia nueva permite representar un estado nuevo sin alterar la instantánea anterior.

Actualización funcional e inmutable

```
setFavoriteIds((currentIds) =>
  toggleFavorite(currentIds, initiativeId),
);
```

Estado mínimo

visibleInitiatives no se guarda con useState porque se obtiene de INITIATIVES, filtros y favoriteIds durante cada renderizado. Duplicar un valor derivable crea dos fuentes que podrían desincronizarse. En

esta colección pequeña el cálculo directo es claro; la optimización solo se estudiaría después de medir una necesidad real.

7 Preparación del proyecto con Vite

Versiones fijadas para reproducir la práctica

El proyecto se verificó con Node.js 24 LTS. Vite 8 requiere Node.js 20.19 o 22.12 en adelante; usar una versión LTS vigente reduce diferencias entre equipos. React 19.3 es la línea estable actual. Bootstrap 5.3 permanece como la línea estable mientras Bootstrap 6 continúa en estado alfa. [R1] [R12] [R13]

Herramienta	Versión del proyecto	Propósito
Node.js y npm	24.19.0 y 11.9.0	Ejecutar scripts y herramientas.
React y react-dom	19.3.1	Componentes, Hooks y raíz del navegador.
Vite y plugin React	8.3.5 y 6.1.2	Desarrollo, transformación y build.
TypeScript	7.0.2	Tipos estrictos y TSX.
Bootstrap	5.3.8	Cuadrícula y estilos base estables.
Vitest y jsdom	5.0.5 y 30.1.3	Pruebas y DOM simulado.
Testing Library	React 16.3.3 y user-event 14.6.8	Pruebas desde la perspectiva del usuario.

Crear la carpeta inicial

1. Comprueba node --version y npm --version. Si Node no cumple el mínimo, instala la línea LTS antes de continuar.
2. Genera el proyecto con la plantilla react-ts y entra en su carpeta.
3. Reemplaza los archivos de configuración por las versiones completas de esta guía.
4. Crea las carpetas components, data, types, utils y test dentro de src.
5. Instala las dependencias para crear package-lock.json; conserva ese archivo en la entrega.

Comando en la raíz del proyecto

```
npm create vite@8.3.5 explorador-iniciativas -- --template react-ts
cd explorador-iniciativas
```

package.json fija cada dependencia y ofrece comandos separados para desarrollo, construcción, pruebas y verificación completa.

package.json Parte 1 de 2

```
{
  "name": "explorador-iniciativas-react",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "preview": "vite preview",
    "test": "vitest run",
    "verify": "npm run build && npm run test"
  },
  "dependencies": {
    "bootstrap": "5.3.8",
    "react": "19.3.1",
```

package.json Parte 2 de 2

```
    "react-dom": "19.3.1"
  },
  "devDependencies": {
    "@testing-library/jest-dom": "6.9.1",
    "@testing-library/react": "16.3.3",
    "@testing-library/user-event": "14.6.8",
    "@types/react": "19.3.0",
    "@types/react-dom": "19.3.0",
    "@vitejs/plugin-react": "6.1.2",
    "jsdom": "30.1.3",
    "typescript": "7.0.2",
    "vite": "8.3.5",
    "vitest": "5.0.5"
  }
}
```

La configuración raíz referencia el código del navegador y la configuración de herramientas.

tsconfig.json

```
{
  "files": [],
  "references": [
    { "path": "./tsconfig.app.json" },
    { "path": "./tsconfig.node.json" }
  ]
}
```

TypeScript trabaja en modo estricto, procesa TSX y no emite archivos porque Vite se ocupa del paquete. noUncheckedIndexedAccess obliga a considerar índices ausentes.

tsconfig.app.json Parte 1 de 2

```
{
  "compilerOptions": {
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "allowJs": false,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "strict": true,
    "forceConsistentCasingInFileNames": true,
```

tsconfig.app.json Parte 2 de 2

```
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true
  },
  "include": ["src"]
}
```

La configuración de herramientas compila vite.config.ts como un módulo moderno y mantiene comprobación estricta.

tsconfig.node.json

```
{
  "compilerOptions": {
    "composite": true,
    "skipLibCheck": true,
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "allowImportingTsExtensions": true,
    "noEmit": true,
    "strict": true
  },
  "include": ["vite.config.ts"]
}
```

El plugin transforma React y la sección test selecciona jsdom, activa globales de Vitest y carga las extensiones de aserciones.

vite.config.ts

```
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: './src/test/setup.ts',
  },
});
```

Las referencias incorporan los tipos de Vite y de los globales utilizados en pruebas.

src/vite-env.d.ts

```
/// <reference types="vite/client" />
/// <reference types="vitest/globals" />
```

Instalar y comprobar

Comando en la raíz del proyecto

```
npm install
npm run build
```

npm install genera package-lock.json. En otra copia limpia utiliza npm ci para instalar exactamente lo registrado. No entregues node_modules ni dist: ambos se reconstruyen.

8 Tipos y conjunto de datos

Tipos cerrados y datos de solo lectura

El archivo define categorías desde una tupla as const y deriva el tipo Category. Esto evita escribir la misma lista dos veces. Las propiedades readonly expresan que los componentes no deben modificar los datos recibidos.

src/types/initiative.ts Parte 1 de 2

```
export const CATEGORIES = [
  'Desarrollo web',
  'Datos',
  'Experiencia de usuario',
] as const;

export type Category = (typeof CATEGORIES)[number];
export type CategoryFilter = 'Todas' | Category;

export interface Initiative {
  readonly id: number;
  readonly title: string;
```

src/types/initiative.ts Parte 2 de 2

```
  readonly description: string;
  readonly category: Category;
  readonly level: 'Inicial' | 'Intermedio';
  readonly durationHours: number;
  readonly skills: readonly string[];
}

export interface InitiativeFilters {
  readonly query: string;
  readonly category: CategoryFilter;
  readonly onlyFavorites: boolean;
}
```

CategoryFilter agrega Todas al conjunto de categorías válidas. InitiativeFilters reúne criterios relacionados. Una cadena arbitraria no puede ocupar category sin que TypeScript lo advierta.

Colección local

La colección contiene seis iniciativas ficticias. Cada objeto satisface Initiative y utiliza ids estables, habilidades como arreglo y valores adecuados a los tipos unión.

src/data/initiatives.ts Parte 1 de 4

```
import type { Initiative } from '../types/initiative';

export const INITIATIVES: readonly Initiative[] = [
  {
    id: 1,
    title: 'Portal de bienestar estudiantil',
    description: 'Organiza recursos y actividades de acompañamiento en una interfaz clara.',
    category: 'Desarrollo web',
    level: 'Inicial',
    durationHours: 12,
    skills: ['HTML semántico', 'React', 'Accesibilidad'],
  },
  {
    id: 2,
    title: 'Tablero de progreso académico',
```

src/data/initiatives.ts Parte 2 de 4

```
    description: 'Presenta indicadores ficticios y ayuda a interpretar tendencias de estudio.',
    category: 'Datos',
    level: 'Intermedio',
    durationHours: 16,
    skills: ['TypeScript', 'Visualización', 'Pruebas'],
  },
  {
    id: 3,
    title: 'Mapa de espacios colaborativos',
    description: 'Facilita la consulta de laboratorios y salas mediante tarjetas reutilizables.',
    category: 'Experiencia de usuario',
    level: 'Inicial',
    durationHours: 10,
    skills: ['Investigación UX', 'Componentes', 'Diseño adaptable'],
  },
```

src/data/initiatives.ts Parte 3 de 4

```
  {
    id: 4,
    title: 'Catálogo de retos de programación',
    description: 'Clasifica ejercicios por nivel y presenta objetivos de aprendizaje concretos.',
    category: 'Desarrollo web',
    level: 'Intermedio',
    durationHours: 18,
    skills: ['React', 'Estado', 'Eventos'],
  },
  {
    id: 5,
    title: 'Observatorio de empleabilidad',
    description: 'Resume datos ficticios sobre habilidades solicitadas en ofertas laborales.',
    category: 'Datos',
    level: 'Intermedio',
```

src/data/initiatives.ts Parte 4 de 4

```
    durationHours: 20,
    skills: ['Arreglos', 'Filtros', 'Comunicación visual'],
  },
  {
    id: 6,
    title: 'Guía de accesibilidad digital',
    description: 'Reúne prácticas para construir experiencias perceptibles y operables.',
    category: 'Experiencia de usuario',
    level: 'Inicial',
    durationHours: 8,
    skills: ['Semántica', 'Teclado', 'Contraste'],
  },
];
```

Contrato de Initiative

Propiedad	Tipo	Uso visible
Id	number	Clave de lista y valor del conjunto de favoritos.
Title	string	Encabezado de la tarjeta y nombre accesible del botón.
description	string	Resumen del propósito.
category	Category	Insignia y filtro cerrado.
level	Inicial o Intermedio	Dato descriptivo.
durationHours	number	Duración presentada con unidad.
skills	readonly string[]	Etiquetas y términos de búsqueda.

9 Reglas puras de búsqueda y favoritos

Separar una regla pura

filterInitiatives recibe todos sus datos y devuelve una colección nueva. No conoce React ni el DOM. toggleFavorite copia el Set antes de agregar o retirar un id. Ambas funciones son fáciles de probar.

src/utils/filterInitiatives.ts Parte 1 de 3

```
import type { Initiative, InitiativeFilters } from '../types/initiative';

export function filterInitiatives(
  initiatives: readonly Initiative[],
  filters: InitiativeFilters,
  favoriteIds: ReadonlySet<number>,
): Initiative[] {
  const normalizedQuery = filters.query.trim().toLocaleLowerCase('es');

  return initiatives.filter((initiative) => {
    const searchableText = [
      initiative.title,
      initiative.description,
      initiative.category,
```

src/utils/filterInitiatives.ts Parte 2 de 3

```
    ...initiative.skills,
  ]
    .join(' ')
    .toLocaleLowerCase('es');

  const matchesQuery =
    normalizedQuery === '' || searchableText.includes(normalizedQuery);
  const matchesCategory =
    filters.category === 'Todas' || initiative.category === filters.category;
  const matchesFavorite =
    !filters.onlyFavorites || favoriteIds.has(initiative.id);

  return matchesQuery && matchesCategory && matchesFavorite;
});
```

src/utils/filterInitiatives.ts Parte 3 de 3

```
}

export function toggleFavorite(
  currentIds: ReadonlySet<number>,
  initiativeId: number,
): Set<number> {
  const nextIds = new Set(currentIds);
  if (nextIds.has(initiativeId)) {
    nextIds.delete(initiativeId);
  } else {
    nextIds.add(initiativeId);
  }
  return nextIds;
}
```

Proceso del filtro

1. Normaliza la búsqueda con trim y minúsculas de configuración española.
2. Construye un texto consultable con título, descripción, categoría y habilidades.
3. Comprueba texto, categoría y favorito como condiciones independientes.
4. Combina las tres condiciones con AND y conserva solo las coincidencias.

Por qué Set para favoritos

Set representa pertenencia y evita duplicados. has consulta si un id está seleccionado; add y delete modifican un conjunto. Como el estado no debe mutarse, toggleFavorite empieza con new Set(currentIds). El número de favoritos se obtiene con size.

Entrada	Resultado esperado
query React	Portal de bienestar y Catálogo de retos.
query teclado	Guía de accesibilidad digital.
categoría Datos	Tablero y Observatorio.
Datos más solo favoritas con ids 2 y 3	Solo id 2.
texto inexistente	Arreglo vacío.

10 Punto de entrada y componente principal

Montar la aplicación

Vite utiliza index.html como entrada. El contenedor root comienza vacío; React administrará su contenido. La etiqueta script type module carga main.tsx.

index.html

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta
      name="description"
      content="Explorador didáctico de iniciativas académicas construido con React"
    />
    <title>Explorador de iniciativas | React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

main.tsx importa estilos, valida la existencia de root y crea una raíz React. StrictMode añade comprobaciones de desarrollo sin producir una etiqueta visible.

src/main.tsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import 'bootstrap/dist/css/bootstrap.min.css';
import './styles.css';
import App from './App';

const rootElement = document.getElementById('root');

if (rootElement === null) {
  throw new Error('No se encontró el elemento #root en index.html.');
}

createRoot(rootElement).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

La verificación contra null evita afirmar al compilador algo que el documento podría incumplir. Si alguien elimina id root de index.html, el mensaje identifica la causa de inmediato.

Componer y coordinar

App posee el estado de la pantalla, deriva visibleInitiatives y compone los hijos. Los componentes reciben únicamente los datos y callbacks que necesitan.

src/App.tsx Parte 1 de 5

```
import { useState } from 'react';
import { EmptyState } from './components/EmptyState';
import { FilterBar } from './components/FilterBar';
import { Hero } from './components/Hero';
import { InitiativeCard } from './components/InitiativeCard';
import { SummaryPanel } from './components/SummaryPanel';
import { INITIATIVES } from './data/initiatives';
import type { CategoryFilter } from './types/initiative';
import { filterInitiatives, toggleFavorite } from './utils/filterInitiatives';

export default function App() {
  const [query, setQuery] = useState('');
  const [category, setCategory] = useState<CategoryFilter>('Todas');
  const [onlyFavorites, setOnlyFavorites] = useState(false);
  const [favoriteIds, setFavoriteIds] = useState<Set<number>>(() => new Set());
```

src/App.tsx Parte 2 de 5

```
  const visibleInitiatives = filterInitiatives(
    INITIATIVES,
    { query, category, onlyFavorites },
    favoriteIds,
  );

  function handleToggleFavorite(initiativeId: number) {
    setFavoriteIds((currentIds) => toggleFavorite(currentIds, initiativeId));
  }

  function clearFilters() {
    setQuery('');
    setCategory('Todas');
    setOnlyFavorites(false);
  }
```

src/App.tsx Parte 3 de 5

```
  return (
    <>
      <Hero total={INITIATIVES.length} />
      <main className="container py-5">
        <FilterBar
          query={query}
          category={category}
          onlyFavorites={onlyFavorites}
          onQueryChange={setQuery}
          onCategoryChange={setCategory}
          onOnlyFavoritesChange={setOnlyFavorites}
          onClear={clearFilters}
        />

        <div className="my-4">
          <SummaryPanel
```

src/App.tsx Parte 4 de 5

```
          visibleCount={visibleInitiatives.length}
          totalCount={INITIATIVES.length}
          favoriteCount={favoriteIds.size}
        />
      </div>

      {visibleInitiatives.length === 0 ? (
        <EmptyState onClear={clearFilters} />
      ) : (
        <section className="row g-4" aria-label="Iniciativas disponibles">
          {visibleInitiatives.map((initiative) => (
            <div className="col-md-6 col-xl-4" key={initiative.id}>
              <InitiativeCard
                initiative={initiative}
                isFavorite={favoriteIds.has(initiative.id)}
                onToggleFavorite={handleToggleFavorite}
```

src/App.tsx Parte 5 de 5

```
              />
            </div>
          ))}
        </section>
      )}
    </main>
    <footer className="container pb-5 text-secondary small">
      Datos ficticios para fines educativos · JavaScript Avanzado
    </footer>
  </>
 );
}
```

Lectura guiada de App

Bloque	Pregunta	Respuesta
useState	¿Qué debe recordarse?	Cuatro decisiones de la interacción.
filterInitiatives	¿Qué debe mostrarse?	Una derivación calculada en el renderizado.
handleToggleFavorite	¿Quién cambia favoritos?	El propietario del estado, App.
clearFilters	¿Qué restablece?	Filtros; conserva favoritos por diseño.
return	¿Cómo se arma la pantalla?	Hero, controles, resumen y una alternativa de contenido.

11 Componentes de encabezado y filtros

Encabezado sin estado

Hero recibe total y presenta contenido. Es un componente de presentación: no crea una copia del total ni consulta la colección.

src/components/Hero.tsx Parte 1 de 2

```
interface HeroProps {
  readonly total: number;
}

export function Hero({ total }: HeroProps) {
  return (
    <header className="hero py-5">
      <div className="container">
        <p className="eyebrow mb-2">LABORATORIO REACT · SEMANA 11</p>
```

src/components/Hero.tsx Parte 2 de 2

```
        <h1 className="display-5 fw-bold mb-3">Explora iniciativas que generan impacto</h1>
        <p className="lead col-lg-8 mb-0">
          Descubre {total} propuestas académicas ficticias, filtra sus características y
          conserva tus favoritas mientras practicas componentes, props y estado.
        </p>
      </div>
    </header>
  );
}
```

Controles administrados por React

FilterBar recibe valor y callback para cada control. value y checked provienen del padre; onChange comunica el siguiente valor. El componente no decide las coincidencias.

src/components/FilterBar.tsx Parte 1 de 5

```
import { CATEGORIES, type CategoryFilter } from '../types/initiative';

interface FilterBarProps {
  readonly query: string;
  readonly category: CategoryFilter;
  readonly onlyFavorites: boolean;
  readonly onQueryChange: (value: string) => void;
  readonly onCategoryChange: (value: CategoryFilter) => void;
  readonly onOnlyFavoritesChange: (value: boolean) => void;
  readonly onClear: () => void;
}

export function FilterBar({
  query,
  category,
```

src/components/FilterBar.tsx Parte 2 de 5

```
  onlyFavorites,
  onQueryChange,
  onCategoryChange,
  onOnlyFavoritesChange,
  onClear,
}: FilterBarProps) {
  return (
    <section className="filter-panel" aria-labelledby="filter-title">
      <div className="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-3">
        <h2 id="filter-title" className="h4 mb-0">Encuentra una iniciativa</h2>
        <button className="btn btn-link px-0" type="button" onClick={onClear}>
          Limpiar filtros
        </button>
      </div>
```

src/components/FilterBar.tsx Parte 3 de 5

```
      <div className="row g-3 align-items-end">
        <div className="col-lg-6">
          <label className="form-label fw-semibold" htmlFor="search">Buscar</label>
          <input
            id="search"
            className="form-control"
            type="search"
            value={query}
            placeholder="Título, habilidad o palabra clave"
            onChange={(event) => onQueryChange(event.target.value)}
          />
        </div>

        <div className="col-md-7 col-lg-4">
          <label className="form-label fw-semibold" htmlFor="category">Categoría</label>
```

src/components/FilterBar.tsx Parte 4 de 5

```
          <select
            id="category"
            className="form-select"
            value={category}
            onChange={(event) => onCategoryChange(event.target.value as CategoryFilter)}
          >
            <option value="Todas">Todas</option>
            {CATEGORIES.map((option) => (
              <option key={option} value={option}>{option}</option>
            ))}
          </select>
        </div>

        <div className="col-md-5 col-lg-2 pb-2">
          <div className="form-check">
```

src/components/FilterBar.tsx Parte 5 de 5

```
          <input
            id="favorites"
            className="form-check-input"
            type="checkbox"
            checked={onlyFavorites}
            onChange={(event) => onOnlyFavoritesChange(event.target.checked)}
          />
          <label className="form-check-label" htmlFor="favorites">Solo favoritas</label>
        </div>
      </div>
    </div>
  </section>
 );
}
```

Controlado y no controlado

Un control es controlado cuando su valor visible procede del estado React y cada cambio llama un manejador. Esto mantiene una fuente de verdad y permite restablecer varios controles. Si escribes value sin onChange, el campo queda bloqueado; si mezclas defaultValue y value sin criterio, el comportamiento resulta confuso.

Control	Valor recibido	Dato emitido
input search	query mediante value	event.target.value
select	category mediante value	event.target.value convertido al tipo cerrado
checkbox	onlyFavorites mediante checked	event.target.checked
botón Limpiar	No conserva valor	Llama onClear

La conversión as CategoryFilter es segura dentro de este select porque todas sus option se generan desde el conjunto cerrado. Si el valor viniera de una API o del almacenamiento, debería validarse en ejecución antes de confiar en él.

12 Tarjetas resumen y estado vacío

Una tarjeta reutilizable

La tarjeta presenta props, recorre habilidades y llama al callback con el id. aria-pressed, el nombre accesible y la estrella cambian con isFavorite.

src/components/InitiativeCard.tsx Parte 1 de 3

```
import type { Initiative } from '../types/initiative';

interface InitiativeCardProps {
  readonly initiative: Initiative;
  readonly isFavorite: boolean;
  readonly onToggleFavorite: (initiativeId: number) => void;
}

export function InitiativeCard({
  initiative,
  isFavorite,
  onToggleFavorite,
}: InitiativeCardProps) {
  return (
    <article className="card initiative-card h-100 border-0 shadow-sm">
      <div className="card-body d-flex flex-column">
```

src/components/InitiativeCard.tsx Parte 2 de 3

```
        <div className="d-flex justify-content-between gap-3 align-items-start">
          <span className="badge category-badge">{initiative.category}</span>
          <button
            className={`favorite-button ${isFavorite ? 'is-favorite' : ''}`}
            type="button"
            aria-pressed={isFavorite}
            aria-label={`${isFavorite ? 'Quitar de' : 'Agregar a'} favoritas: ${initiative.title}`}
            onClick={() => onToggleFavorite(initiative.id)}
          >
            <span aria-hidden="true">{isFavorite ? '★' : '☆'}</span>
          </button>
        </div>

        <h3 className="h5 mt-3">{initiative.title}</h3>
        <p className="text-secondary flex-grow-1">{initiative.description}</p>
```

src/components/InitiativeCard.tsx Parte 3 de 3

```
        <dl className="row small mb-3">
          <dt className="col-5">Nivel</dt>
          <dd className="col-7">{initiative.level}</dd>
          <dt className="col-5">Duración</dt>
          <dd className="col-7 mb-0">{initiative.durationHours} horas</dd>
        </dl>

        <ul className="skill-list" aria-label={`Habilidades de ${initiative.title}`}>
          {initiative.skills.map((skill) => <li key={skill}>{skill}</li>)}
        </ul>
      </div>
    </article>
  );
}
```

El clic no ejecuta onToggleFavorite inmediatamente durante el renderizado. La prop onClick recibe una función flecha que se ejecutará después. Escribir onClick={onToggleFavorite(initiative.id)} llamaría la función al renderizar y pasaría su resultado como manejador.

Resumen anunciado

SummaryPanel recibe cantidades ya calculadas. aria-live polite permite anunciar cambios sin interrumpir al usuario. La singularización evita mostrar 1 favoritas.

src/components/SummaryPanel.tsx Parte 1 de 2

```
interface SummaryPanelProps {
  readonly visibleCount: number;
  readonly totalCount: number;
  readonly favoriteCount: number;
}

export function SummaryPanel({
  visibleCount,
  totalCount,
  favoriteCount,
}: SummaryPanelProps) {
```

src/components/SummaryPanel.tsx Parte 2 de 2

```
  return (
    <div className="summary-panel" aria-live="polite">
      <p className="mb-0">
        Mostrando <strong>{visibleCount}</strong> de <strong>{totalCount}</strong>{' '}
        iniciativas · <strong>{favoriteCount}</strong>{' '}
        {favoriteCount === 1 ? 'favorita' : 'favoritas'}
      </p>
    </div>
  );
}
```

Estado vacío recuperable

EmptyState explica el resultado, evita una página aparentemente rota y ofrece restablecer los filtros. El símbolo decorativo se oculta a tecnologías de asistencia.

src/components/EmptyState.tsx

```
interface EmptyStateProps {
  readonly onClear: () => void;
}

export function EmptyState({ onClear }: EmptyStateProps) {
  return (
    <div className="empty-state text-center" role="status">
      <p className="display-6 mb-2" aria-hidden="true">⌕</p>
      <h2 className="h4">No encontramos coincidencias</h2>
      <p className="text-secondary">Prueba otra palabra o restablece los filtros.</p>
      <button className="btn btn-primary" type="button" onClick={onClear}>
        Mostrar todas
      </button>
    </div>
  );
}
```

Principio de propiedad

SummaryPanel y EmptyState no guardan datos duplicados. App es el propietario más cercano que necesita coordinar controles, tarjetas y resumen. Elevar el estado permite que varios hijos observen una misma fuente. [R10]

13 Listas condicionales y claves estables

Renderizar una colección

map convierte cada objeto visible en un elemento React. La prop key permite reconocer la identidad entre renderizados. El id de los datos es estable; el índice de la posición no lo sería si la colección se filtra, reordena o amplía. key no llega automáticamente como prop al componente. [R6]

Patrón reducido de lista con una clave estable

```
{visibleInitiatives.map((initiative) => (
  <div key={initiative.id}>
    <InitiativeCard initiative={initiative} />
  </div>
))}
```

Renderizado condicional

La expresión ternaria elige entre EmptyState y la lista. Ambas ramas describen resultados completos. Dentro de la tarjeta, otro ternario selecciona estrella llena o vacía. Para una condición que solo añade contenido, también podría emplearse && con cuidado de no renderizar valores numéricos inesperados. [R7]

Condición	Rama visible
visibleInitiatives.length es 0	Explicación del vacío y botón Mostrar todas.
visibleInitiatives.length es mayor que 0	Sección con cuadrícula de tarjetas.
isFavorite es true	Estrella llena, clase is-favorite y aria-pressed true.
favoriteCount es 1	Texto singular favorita.

No mezclar filtro con marcado

App calcula visibleInitiatives antes del return. Esto mantiene el marcado legible y permite probar la regla por separado. No se necesita un useEffect para filtrar: el valor es una derivación síncrona de información ya disponible.

14 Diseño con Bootstrap y CSS accesible

Bootstrap como capa de estilo

Bootstrap se importa desde node_modules y aporta cuadrícula, espaciado, formularios y utilidades. No se carga su JavaScript porque el proyecto no utiliza componentes interactivos de Bootstrap. React controla las interacciones propias. [R13]

Los estilos propios construyen la identidad visual, estados de foco, tarjetas, insignias y una alternativa de movimiento reducido. Bootstrap resuelve la cuadrícula y las bases; el CSS del proyecto expresa el diseño particular.

src/styles.css Parte 1 de 4

```
:root {
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  color: #17233b;
  background: #f5f7fb;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

body { min-width: 320px; min-height: 100vh; margin: 0; background: #f5f7fb; }
button, input, select { font: inherit; }

.hero {
  color: #fff;
  background:
    radial-gradient(circle at 82% 18%, rgba(70, 210, 188, .3), transparent 28%),
```

src/styles.css Parte 2 de 4

```
    linear-gradient(128deg, #111f3d 0%, #214b7a 56%, #18756e 100%);
}

.eyebrow { font-size: .78rem; font-weight: 800; letter-spacing: .14em; color: #9ff2df; }

.filter-panel, .summary-panel, .empty-state {
  border: 1px solid #dfe5ee;
  border-radius: 1rem;
  background: #fff;
}

.filter-panel { padding: 1.5rem; box-shadow: 0 .6rem 1.8rem rgba(27, 45, 75, .06); }
.summary-panel { padding: .9rem 1.1rem; border-left: .35rem solid #1d8a7b; }
.empty-state { padding: 3.5rem 1.5rem; }
```

src/styles.css Parte 3 de 4

```
.initiative-card { border-radius: 1rem; transition: transform 160ms ease, box-shadow 160ms ease; }
.initiative-card:hover { transform: translateY(-3px); box-shadow: 0 1rem 2.2rem rgba(27, 45, 75, .13)
!important; }
.initiative-card dd { margin-bottom: .25rem; }

.category-badge { color: #155a52; background: #d9f2ed; font-weight: 700; }

.favorite-button {
  display: grid;
  width: 2.6rem;
  height: 2.6rem;
  place-items: center;
  border: 1px solid #c9d2df;
  border-radius: 50%;
  color: #53657c;
  background: #fff;
```

src/styles.css Parte 4 de 4

```
  font-size: 1.45rem;
  line-height: 1;
}

.favorite-button:hover, .favorite-button:focus-visible { border-color: #9a6500; color: #7a5000; }
.favorite-button.is-favorite { border-color: #d69a1c; color: #7a5000; background: #fff0c7; }

.skill-list { display: flex; flex-wrap: wrap; gap: .4rem; margin: 0; padding: 0; list-style: none; }
.skill-list li { padding: .25rem .55rem; border-radius: 999px; color: #33445e; background: #eef2f7; font-
size: .78rem; }

:focus-visible { outline: .2rem solid #f2a900; outline-offset: .18rem; }

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { scroll-behavior: auto !important; transition-duration: .01ms !important; }
}
```

Semántica y accesibilidad aplicadas

Decisión	Beneficio
header main footer y article	La estructura comunica regiones y unidades de contenido.
label con htmlFor	El nombre se asocia explícitamente con cada control.
button real	Favorito y limpieza funcionan con teclado de forma nativa.
aria-pressed	Comunica el estado alternable sin depender de la estrella o el color.
aria-live polite	Los cambios de cantidades pueden anunciarse sin interrumpir.
focus-visible	La ubicación del teclado permanece perceptible.
prefers-reduced-motion	Reduce transiciones para quien expresa esa preferencia.

Comprobación manual obligatoria

La compilación no comprueba contraste, orden de foco ni claridad visual. Recorre la aplicación con Tab y Shift+Tab, activa botones con Enter o Espacio, amplía el texto y reduce el ancho. La estrella no debe ser el único indicador: el nombre accesible y aria-pressed expresan el mismo estado.

15 Pruebas automatizadas

Configurar el entorno

El archivo añade aserciones semánticas como toBeInTheDocument y toHaveAttribute a Vitest.

src/test/setup.ts

```
import '@testing-library/jest-dom/vitest';
```

Probar reglas puras

Siete pruebas comprueban búsqueda, combinación, inmutabilidad y cambio de favoritos sin crear un componente.

src/utils/filterInitiatives.test.ts Parte 1 de 4

```
import { describe, expect, it } from 'vitest';
import { INITIATIVES } from '../data/initiatives';
import { filterInitiatives, toggleFavorite } from './filterInitiatives';

describe('filterInitiatives', () => {
  it('returns every initiative with the initial filters', () => {
    const result = filterInitiatives(
      INITIATIVES,
      { query: '', category: 'Todas', onlyFavorites: false },
      new Set(),
    );
    expect(result).toHaveLength(6);
  });

  it('normalizes spaces and uppercase characters', () => {
    const result = filterInitiatives(
```

src/utils/filterInitiatives.test.ts Parte 2 de 4

```
      INITIATIVES,
      { query: '  REACT ', category: 'Todas', onlyFavorites: false },
      new Set(),
    );
    expect(result.map(({ id }) => id)).toEqual([1, 4]);
  });

  it('searches title, description, category and skills', () => {
    const result = filterInitiatives(
      INITIATIVES,
      { query: 'teclado', category: 'Todas', onlyFavorites: false },
      new Set(),
    );
    expect(result[0]?.title).toBe('Guía de accesibilidad digital');
  });
```

src/utils/filterInitiatives.test.ts Parte 3 de 4

```
  it('combines category and favorite filters', () => {
    const result = filterInitiatives(
      INITIATIVES,
      { query: '', category: 'Datos', onlyFavorites: true },
      new Set([2, 3]),
    );
    expect(result.map(({ id }) => id)).toEqual([2]);
  });

  it('does not mutate the source collection', () => {
    const snapshot = JSON.stringify(INITIATIVES);
    filterInitiatives(
      INITIATIVES,
      { query: 'react', category: 'Todas', onlyFavorites: false },
      new Set(),
    );
```

src/utils/filterInitiatives.test.ts Parte 4 de 4

```
    expect(JSON.stringify(INITIATIVES)).toBe(snapshot);
  });
});

describe('toggleFavorite', () => {
  it('adds an id without mutating the original Set', () => {
    const original = new Set([1]);
    const result = toggleFavorite(original, 2);
    expect([...result]).toEqual([1, 2]);
    expect([...original]).toEqual([1]);
  });

  it('removes an id that is already selected', () => {
    expect([...toggleFavorite(new Set([1, 2]), 1)]).toEqual([2]);
  });
});
```

Probar el contrato de una tarjeta

Cuatro pruebas observan contenido, estado accesible y callback. La prueba interactiva utiliza userEvent para aproximarse al comportamiento de una persona.

src/components/InitiativeCard.test.tsx Parte 1 de 4

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { describe, expect, it, vi } from 'vitest';
import { INITIATIVES } from '../data/initiatives';
import { InitiativeCard } from './InitiativeCard';

const initiative = INITIATIVES[0]!;

describe('InitiativeCard', () => {
  it('renders the initiative data received through props', () => {
    render(
      <InitiativeCard initiative={initiative} isFavorite={false} onToggleFavorite={() => {}} />,
    );
```

src/components/InitiativeCard.test.tsx Parte 2 de 4

```
    expect(screen.getByRole('heading', { name: initiative.title })).toBeInTheDocument();
    expect(screen.getByText('12 horas')).toBeInTheDocument();
  });

  it('announces that an unselected initiative can be added', () => {
    render(
      <InitiativeCard initiative={initiative} isFavorite={false} onToggleFavorite={() => {}} />,
    );
    expect(screen.getByRole('button', { name: /agregar a favoritas/i })).toHaveAttribute(
      'aria-pressed',
      'false',
    );
  });
```

src/components/InitiativeCard.test.tsx Parte 3 de 4

```
  it('announces that a selected initiative can be removed', () => {
    render(
      <InitiativeCard initiative={initiative} isFavorite onToggleFavorite={() => {}} />,
    );
    expect(screen.getByRole('button', { name: /quitar de favoritas/i })).toHaveAttribute(
      'aria-pressed',
      'true',
    );
  });

  it('sends the initiative id when the user activates the button', async () => {
    const user = userEvent.setup();
```

src/components/InitiativeCard.test.tsx Parte 4 de 4

```
    const onToggleFavorite = vi.fn();
    render(
      <InitiativeCard
        initiative={initiative}
        isFavorite={false}
        onToggleFavorite={onToggleFavorite}
      />,
    );
    await user.click(screen.getByRole('button', { name: /agregar a favoritas/i }));
    expect(onToggleFavorite).toHaveBeenCalledOnce();
    expect(onToggleFavorite).toHaveBeenCalledWith(initiative.id);
  });
});
```

Probar recorridos de la aplicación

Siete pruebas recorren búsqueda, categoría, favoritos, vacío y limpieza de filtros mediante roles y nombres accesibles.

src/App.test.tsx Parte 1 de 4

```
import { render, screen, within } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { describe, expect, it } from 'vitest';
import App from './App';

describe('App', () => {
  it('renders the six initiatives and the initial summary', () => {
    render(<App />);
    expect(screen.getAllByRole('article')).toHaveLength(6);
    expect(screen.getByText(/mostrando/i)).toHaveTextContent('Mostrando 6 de 6 iniciativas');
  });

  it('filters initiatives while the user writes', async () => {
    const user = userEvent.setup();
    render(<App />);
    await user.type(screen.getByRole('searchbox', { name: /buscar/i }), 'React');
    expect(screen.getAllByRole('article')).toHaveLength(2);
```

src/App.test.tsx Parte 2 de 4

```
  expect(screen.queryByText('Observatorio de empleabilidad')).not.toBeInTheDocument();
 });

 it('filters by category', async () => {
  const user = userEvent.setup();
  render(<App />);
  await user.selectOptions(screen.getByRole('combobox', { name: /categoría/i }), 'Datos');
  expect(screen.getAllByRole('article')).toHaveLength(2);
 });

 it('adds and removes a favorite', async () => {
  const user = userEvent.setup();
  render(<App />);
  const card = screen.getByRole('heading', { name: /portal de bienestar/i }).closest('article')!;
  await user.click(within(card).getByRole('button', { name: /agregar a favoritas/i }));
  expect(screen.getByText(/mostrando/i)).toHaveTextContent('1 favorita');
  await user.click(within(card).getByRole('button', { name: /quitar de favoritas/i }));
```

src/App.test.tsx Parte 3 de 4

```
  expect(screen.getByText(/mostrando/i)).toHaveTextContent('0 favoritas');
 });

 it('shows only the initiatives selected as favorites', async () => {
  const user = userEvent.setup();
  render(<App />);
  await user.click(screen.getAllByRole('button', { name: /agregar a favoritas/i })[0]!);
  await user.click(screen.getByRole('checkbox', { name: /solo favoritas/i }));
  expect(screen.getAllByRole('article')).toHaveLength(1);
 });

 it('shows an empty state and restores every initiative', async () => {
  const user = userEvent.setup();
  render(<App />);
  await user.type(screen.getByRole('searchbox', { name: /buscar/i }), 'sin coincidencias');
  expect(screen.getByRole('heading', { name: /no encontramos/i })).toBeInTheDocument();
  await user.click(screen.getByRole('button', { name: /mostrar todas/i }));
```

src/App.test.tsx Parte 4 de 4

```
  expect(screen.getAllByRole('article')).toHaveLength(6);
 });

 it('clears combined filters without removing favorites', async () => {
  const user = userEvent.setup();
  render(<App />);
  await user.click(screen.getAllByRole('button', { name: /agregar a favoritas/i })[0]!);
  await user.type(screen.getByRole('searchbox', { name: /buscar/i }), 'portal');
  await user.selectOptions(screen.getByRole('combobox', { name: /categoría/i }), 'Desarrollo web');
  await user.click(screen.getByRole('button', { name: /limpiar filtros/i }));
  expect(screen.getAllByRole('article')).toHaveLength(6);
  expect(screen.getByText(/mostrando/i)).toHaveTextContent('1 favorita');
 });
});
```

Distribución de las 18 pruebas

Archivo	Pruebas	Cobertura principal
filterInitiatives.test.ts	7	Reglas puras, combinación e inmutabilidad.
InitiativeCard.test.tsx	4	Props, aria-pressed y callback con id.

Archivo	Pruebas	Cobertura principal
App.test.tsx	7	Estado, controles, renderizado y recuperación.
Total	18	Comportamiento esencial del laboratorio.

Testing Library prioriza consultas semejantes a cómo una persona encuentra elementos: roles, etiquetas y texto visible. Una prueba que solo busca clases internas puede aprobar aunque la experiencia accesible se haya roto. [R16]

16 Ejecución y validación manual

Comandos principales

Comando	Propósito	Resultado esperado
npm run dev	Servidor de desarrollo.	URL local y recarga al guardar.
npm run build	Comprobación TypeScript y paquete.	Carpeta dist sin errores.
npm run preview	Previsualizar el build.	Servidor local del paquete generado.
npm run test	Ejecutar pruebas una vez.	3 archivos y 18 pruebas aprobadas.
npm run verify	Build seguido de pruebas.	Verificación integral reproducible.

Comando en la raíz del proyecto

```
npm run verify
```

La implementación incluida superó la compilación de producción con Vite 8.3.5 y las 18 pruebas con Vitest 5.0.5. Repite el comando en tu equipo y conserva la salida completa como evidencia.

Matriz de validación manual

Caso	Acción	Resultado esperado
M1	Abrir la aplicación.	Seis tarjetas, resumen 6 de 6 y cero favoritas.
M2	Buscar React.	Aparecen dos tarjetas y el resumen indica 2 de 6.
M3	Limpiar y elegir Datos.	Aparecen Tablero y Observatorio.
M4	Marcar Portal como favorita.	Estrella llena, aria-pressed true y resumen con 1 favorita.
M5	Activar Solo favoritas.	Solo aparece la tarjeta marcada.
M6	Buscar un texto inexistente.	Estado vacío con acción Mostrar todas.
M7	Pulsar Mostrar todas.	Se restablecen filtros sin perder favoritos.
M8	Usar únicamente teclado.	Foco visible y todos los controles operables.

Caso	Acción	Resultado esperado
M9	Reducir el ancho a 360 px.	Controles y tarjetas se apilan sin desbordamiento horizontal.
M10	Recargar la página.	Los filtros y favoritos vuelven al estado inicial; aún no hay persistencia.

17 Diagnóstico de errores frecuentes

Método de diagnóstico

1. Lee el primer error y localiza archivo, línea y categoría: TypeScript, transformación, ejecución o prueba.
2. Reduce el problema a una interacción o componente y evita cambiar varios archivos a la vez.
3. Inspecciona terminal y consola del navegador; compáralas con el estado visible.
4. Corrige la causa, ejecuta la prueba afectada y después npm run verify.
5. Explica por qué la corrección funciona; no ocultes un tipo con any ni suprimas el error sin análisis.

Errores frecuentes

Síntoma	Causa probable	Corrección
Pantalla en blanco.	Excepción o root ausente.	Lee consola y verifica index.html y main.tsx.
JSX no compila.	Etiqueta abierta, raíz múltiple o archivo .ts.	Cierra etiquetas, agrupa nodos y usa .tsx.
class genera advertencia.	Se copió HTML literal.	Utiliza className.
Componente no aparece.	Nombre minúsculo o falta return.	Usa PascalCase y devuelve TSX.
Campo no permite escribir.	value no tiene onChange funcional.	Actualiza el estado desde event.target.value.
Favorito no cambia.	Se mutó el Set actual.	Crea new Set y devuelve la copia mediante el setter.
Callback corre al cargar.	Se llamó la función en onClick.	Pasa una función: () => callback(id).
Mensaje key ausente.	Elemento creado por map sin identidad.	Añade key con initiative.id al elemento superior.
Resultados desfasados.	Se duplicó la lista visible como estado.	Deriva la colección en el renderizado.
Checkbox no responde.	Se leyó value en vez de checked.	Usa event.target.checked.
Bootstrap no aplica.	Import ausente o paquete no instalado.	Revisa main.tsx, package.json y npm install.

Síntoma	Causa probable	Corrección
Prueba no encuentra control.	Falta label o nombre accesible.	Corrige la semántica antes de cambiar la consulta.
Dos logs en desarrollo.	StrictMode repite ciertas ejecuciones.	Busca efectos impuros; no retires StrictMode como primera solución.

18 Práctica guiada de comprensión

Experimento A: sigue el dato

1. Busca la tarjeta Portal de bienestar estudiantil en la colección y anota su id.
2. Localiza dónde App convierte la colección visible en InitiativeCard.
3. Sigue initiative hasta el h3 y favoriteIds.has hasta aria-pressed.
4. Activa la estrella y explica el recorrido del callback hasta setFavoriteIds.
5. Relaciona el estado nuevo con el siguiente renderizado y el cambio de SummaryPanel.

Experimento B: distingue estado y derivación

Valor	¿Estado?	Justificación
query	Sí	La entrada del usuario cambia y debe recordarse.
category	Sí	La selección cambia por interacción.
favoriteIds	Sí	La preferencia cambia y alimenta varias partes.
visibleInitiatives	No	Se calcula con datos y estado existentes.
favoriteIds.size	No	Es una propiedad del estado actual.
INITIATIVES.length	No	Es una propiedad de datos constantes.

Experimento C: predice antes de ejecutar

1. Predice cuántas tarjetas habrá al buscar datos y elegir Desarrollo web. Luego compruébalo.
2. Predice qué conserva Limpiar filtros después de marcar dos favoritas. Justifica con clearFilters.
3. Predice qué ocurre si toggleFavorite retorna currentIds después de mutarlo. Relaciónalo con identidad.
4. Predice el nombre accesible del botón antes y después de marcar una iniciativa.

19 Reto individual de ampliación

Reto: ordenar sin mutar

Añade un select Ordenar con tres opciones: Orden original, Menor duración y Mayor duración. El orden debe combinarse con los filtros existentes y no debe modificar INITIATIVES. Define un tipo cerrado, agrega el criterio a InitiativeFilters y ordena una copia del resultado.

1. El control es administrado por App y tiene label explícito.
2. Limpiar filtros restablece Orden original y conserva favoritos.
3. La función pura devuelve un arreglo nuevo antes de sort.
4. Añade al menos tres pruebas: ascendente, descendente e inmutabilidad.
5. Explica por qué sort sobre el arreglo original sería un defecto.

Reto de autonomía: panel de selección

Añade una acción Ver detalle a cada tarjeta. Solo una iniciativa puede estar seleccionada. Presenta un panel debajo de la cuadrícula con título, descripción, habilidades y un botón Cerrar. Este reto sigue usando estado local: no incorpora rutas ni solicitudes HTTP.

1. App conserva selectedId como number o null y deriva la iniciativa seleccionada.
2. La tarjeta emite el id mediante un callback; no busca datos globales.
3. El panel posee un encabezado y el foco se gestiona de forma comprensible.
4. Una prueba abre una iniciativa y otra la cierra.

Extensión opcional

Persistir favoritos en localStorage queda como extensión porque requiere sincronizar React con una API externa al renderizado y se desarrollará con mayor profundidad al estudiar efectos y Hooks. Si la implementas, valida los datos leídos, maneja JSON inválido y documenta por qué esa sincronización no es un cálculo puro.

20 Preguntas de comprobación

Pregunta	Orientación para comprobar tu respuesta
1 Qué papel cumple React	Describe interfaces mediante componentes y actualiza el DOM desde datos.
2 Qué papel cumple Vite	Sirve desarrollo, transforma módulos y produce el build.
3 Por qué el archivo es TSX	Combina sintaxis JSX con comprobación TypeScript.
4 Qué diferencia props y estado	Props llegan del padre; estado es memoria propia del propietario.
5 Por qué un hijo llama un callback	Solicita un cambio al componente que posee el estado.
6 Por qué no se muta favoriteIds	Debe producirse un estado nuevo y conservar la instantánea anterior.

Pregunta	Orientación para comprobar tu respuesta
7 Por qué visibleInitiatives no es estado	Puede calcularse desde datos, filtros y favoritos.
8 Qué hace key	Identifica un elemento de lista entre renderizados.
9 Por qué no usar el índice	La posición cambia al filtrar, ordenar o insertar.
10 Qué hace un control controlado	Su valor procede del estado y onChange solicita actualizarlo.
11 Qué comunica aria-pressed	El estado actual de un botón alternable.
12 Qué demuestra StrictMode	Ayuda a revelar impurezas de desarrollo; no duplica la interfaz final.
13 Qué prueba la función pura	Reglas de búsqueda y favoritos sin renderizar React.
14 Qué no prueba el build	No valida por sí solo teclado, contraste ni claridad visual.
15 Cuándo usarías estado elevado	Cuando varios hijos necesitan coordinar el mismo dato.

Responde al menos ocho preguntas señalando una línea, una interacción o una prueba del proyecto. Una respuesta completa conecta el concepto con una consecuencia observable.

21 Referencias

R1 React Versiones https://react.dev/versions

R2 React Escribir marcado con JSX https://react.dev/learn/writing-markup-with-jsx

R3 React Tu primer componente https://react.dev/learn/your-first-component

R4 React Pasar props a un componente https://react.dev/learn/passing-props-to-a-component

R5 React Responder a eventos https://react.dev/learn/responding-to-events

R6 React Renderizar listas https://react.dev/learn/rendering-lists

R7 React Renderizado condicional https://react.dev/learn/conditional-rendering

R8 React useState https://react.dev/reference/react/useState

R9 React Reglas de Hooks https://react.dev/reference/rules/rules-of-hooks

R10 React Compartir estado entre componentes https://react.dev/learn/sharing-state-between-components

R11 React StrictMode https://react.dev/reference/react/StrictMode

R12 Vite Guía de inicio https://vite.dev/guide/

R13 Bootstrap Integración con Vite https://getbootstrap.com/docs/5.3/getting-started/vite/

R14 TypeScript JSX https://www.typescriptlang.org/docs/handbook/jsx.html

R15 WAI ARIA Button Pattern https://www.w3.org/WAI/ARIA/apg/patterns/button/

R16 Testing Library Consultas https://testing-library.com/docs/queries/about/

R17 Node.js Versiones oficiales https://nodejs.org/en/about/previous-releases